

IK141 Struktur Data
Pendahuluan Struktur Data
Stack



Di Susun Oleh :
Deden Ahmad Jamil
2501518

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER
FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM

1. Implementasi dan Hasil

Implementasi dan Hasil

Ss code program nomor 1:

a. **Main:**

```
1 // nama : deden ahmad jamil
2 // nim : 2501518
3
4 #include "header.h"
5
6 int main() {
7     Stack tumpukan;
8     init(&tumpukan);
9
10    char ekspresi[MAX];
11    int seimbang = 1; // kita asumsiin benar dulu di awal
12
13    printf("Masukkan ekspresi: ");
14    scanf("%[^\n]s", ekspresi);
15
16    // loop buat ngecek setiap karakter dari kiri ke kanan
17    for (int i = 0; i < strlen(ekspresi); i++) {
18        // kalau ketemu kurung buka, langsung masukin (Push)
19        if (ekspresi[i] == '(') {
20            push(&tumpukan, '(');
21        }
22        // kalau ketemu kurung tutup, kita keluarin pasangannya (Pop)
23        else if (ekspresi[i] == ')') {
24            if (isEmpty(&tumpukan)) {
25                seimbang = 0; // ada tutup tapi gak ada bukanya artinya ga seimbang
26                break;
27            }
28            pop(&tumpukan);
29        }
30    }
```



```
1  // di akhir stack harus kosong biar bisa dibilang seimbang
2  if (seimbang && isEmpty(&tumpukan)) {
3      printf("Kurung seimbang.\n");
4  } else {
5      printf("Kurung tidak seimbang.\n");
6  }
7
8  return 0;
9  }
```

b. **header.h:**



```
1 // nama : deden ahmad jamil
2 // nim : 2501518
3
4 #include <stdio.h>
5 #include <string.h>
6
7 #define MAX 100 // inisialisasi batas max
8
9 // struktur stack
10 typedef struct {
11     char data[MAX];
12     int top;
13 } Stack;
14
```



```
1 // inisialisasi stack biar gak random isinya
2 void init(Stack *s) {
3     s->top = -1; // Top -1 artinya stack masih kosong melompong
4 }
5
6 // fungsi ngecek stack kosong atau nggak
7 int isEmpty(Stack *s) {
8     return s->top == -1; // kalau top masih -1 berarti kosong
9 }
```

```

1 // fungsi buat masukin kurung ke stack
2 void push(Stack *s, char c) {
3     if (s->top < MAX - 1) {
4         s->top++;
5         s->data[s->top] = c; // masukin karakter ke puncak
6     }
7 }
8
9 // fungsi buat ngeluarin kurung dari stack
10 char pop(Stack *s) {
11     if (!isEmpty(s)) {
12         return s->data[s->top--]; // Ambil datanya terus topnya turun
13     }
14     return '\0';
15 }

```

Output:

a. Output kalo ga seimbang

```

deden@Dden MINGW64 ~/OneDrive/Dokumen/matakuliah(semester2)/strukdat/TUGAS_LATIHAN_PRAKTIKUM/LP7/no1
$ ./main.exe
Masukkan ekspresi: (6*1(8+1)
Kurung tidak seimbang.

```

b. Output kalo seimbang

```

deden@Dden MINGW64 ~/OneDrive/Dokumen/matakuliah(semester2)/strukdat/TUGAS_LATIHAN_PRAKTIKUM/LP7/no1
$ ./main.exe
Masukkan ekspresi: (3*2(-1+2))
Kurung seimbang.

```

Ss code program nomor 2:

a. Main:



```
1 // nama: deden ahmad jamil
2 // nim: 2501518
3
4 #include "header_no2.h"
5
6 int main() {
7     Stack tumpukan;
8     init(&tumpukan);
9
10    char kalimat[MAX];
11    printf("Masukkan kalimat: ");
12    scanf("%[^\n]s", kalimat);
13
14    printf("Kalimat setelah dibalik: ");
15
16    // loop buat baca kalimat huruf demi huruf dari kiri
17    for (int i = 0; i <= strlen(kalimat); i++) {
18        // kalau belum ketemu spasi dan belum akhir kalimat push terus
19        if (kalimat[i] != ' ' && kalimat[i] != '\0') {
20            push(&tumpukan, kalimat[i]);
21        }
22        // kalau ketemu spasi atau akhir kalimat waktunya kita balikin katanya pop
23        else {
24            while (!isEmpty(&tumpukan)) {
25                printf("%c", pop(&tumpukan));
26            }
27            if (kalimat[i] == ' ') {
28                printf(" "); // tambah spasi lagi biar antar kata gak nempel
29            }
30        }
31    }
32    printf("\n");
33
34    return 0;
35 }
```

b. Header dan fungsi:



```
1 // nama : deden ahmad jamil
2 // nim : 2501518
3
4 #include <stdio.h>
5 #include <string.h>
6
7 #define MAX 100 // batas maksimal
8
9 // struktur stack buat nampung huruf tiap kata
10 typedef struct {
11     char data[MAX];
12     int top;
13 } Stack;
```




```
1 // inisialisasi stack biar siap dipake
2 void init(Stack *s) {
3     s->top = -1; // -1 artinya stack masih kosong melompong
4 }
5
6 // fungsi ngecek stack kosong atau nggak
7 int isEmpty(Stack *s) {
8     return s->top == -1; // Kalau top -1 berarti gak ada isinya
9 }
```

```

1 // Masukin huruf ke stack
2 void push(Stack *s, char c) {
3     if (s->top < MAX - 1) { // cek dulu jangan sampe penuh
4         s->top++;
5         s->data[s->top] = c; // huruf masuk ke tumpukan
6     }
7 }
8
9 // ngeluarin huruf dari stack
10 char pop(Stack *s) {
11     if (!isEmpty(s)) { // cek jangan sampe kosong
12         return s->data[s->top--]; // Ambil datanya terus topnya turun
13     }
14     return '\0';
15 }

```

Output:

```

deden@Dden MINGW64 ~/OneDrive/Dokumen/matakuliah(semester2)/strukdat/TUGAS_LATIHAN_PRAKTIKUM/LP7/no2
$ ./main_no2.exe
Masukkan kalimat: Aku Sayang Mantan
Kalimat setelah dibalik: uKA gnayaS natnaM

```

2. Kesimpulan

Kesimpulan

Praktikum Modul 7 ini ngajarin aku kalau *stack* itu ibarat tumpukan piring yang pakai prinsip siapa yang terakhir ditaruh, dia yang pertama diambil (LIFO). Meskipun jujur pengerjaannya kerasa lumayan berat karena aku lagi nggak fokus, aku jadi paham kalau kita nggak bisa asal masukin atau ngambil data tanpa ngecek tumpukannya lagi penuh

(*isFull*) atau malah kosong (***isEmpty***). Kunci utamanya ada di variabel **top** yang dimulai dari **-1** buat nandain kalau tumpukannya bener-bener belum ada isinya. Intinya, logika tumpukan ini ngebantu banget buat nyelesain masalah simpel kayak ngebalik kata atau mastiin kurung di kode program udah pas pasangannya